НИТУ «МИСИС»

Кафедра инженерной кибернетики

ОТЧЕТ

ПО

ЛАБОРАТОРНОЙ РАБОТЕ №8.3

«Измерение пропускной способности памяти видеокарт

с поддержкой технологии CUDA»

учебная дисциплина «Параллельные вычисления»

Студент: Голубев И. М., БПМ-22-ПО-1

Преподаватель: Ватутин Э. И.

Оценка:

Дата защиты: 27.04.2026

2026 г.

Введение

Основная цель работы:

Научиться измерять пропускную способность памяти GPU при различных типах и направлениях передачи данных.

Описание решаемой задачи

1. Выделить блоки памяти одинакового размера в оперативной памяти и глобальной памяти видеокарты. Скопировать содержимое блоков между:

- двумя буферами в оперативной памяти;

- между оперативной памятью и глобальной памятью видеокарты в направлении CPU GPU и GPU CPU с использованием обычной и page-locked памяти (должно быть 4 различных варианта копирования);

- между двумя буферами в глобальной памяти видеокарты.

2. Убедиться в том, что копирование происходит корректно.

3. Измерить время копирования и, зная размер копируемого блока, определить пропускную способность (в ГБ/с) для каждого типа копирования. Сделать выводы о скорости копирования в различных режимах.

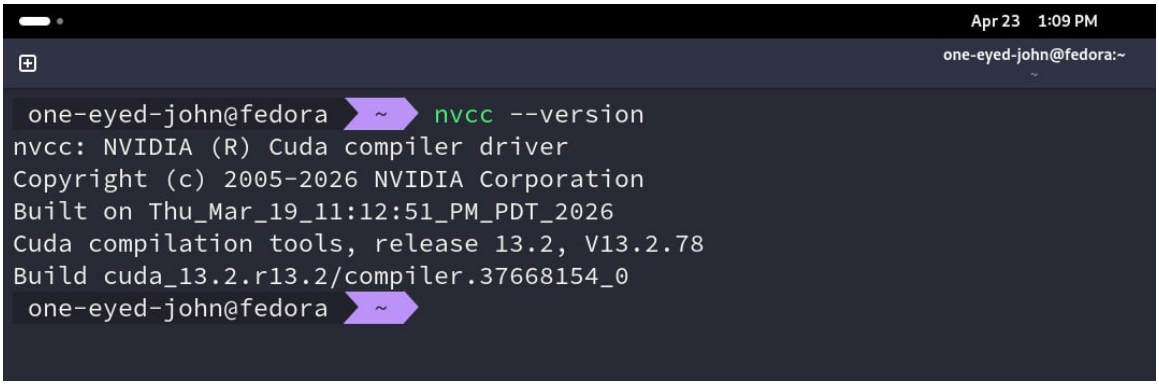
4. В отчет включить краткое описание типа и параметров видеокарты и процессора.

Выбранные средства для разработки

Для выполнения лабораторной работы использовались следующие программные средства и технологии:

- Язык программирования: C++17
- Среда разработки: Zed
- Операционная система: Fedora Linux
- Процессор: Intel(R) Core(TM) i5-10300H CPU @ 2.50GHz
- Размер RAM: 15Gi

- Компиляция выполняется средствами утилиты nvcc



Листинг кода программы

Код доступен в удалённом Github репозитории по ссылке:

https://github.com/g0lubew-iv/parallel-comp/blob/main/lab08/lab08_3.cpp

https://github.com/g0lubew-iv/parallel-comp/blob/main/lab08/lab08_3.cu

Файл lab08_3.cpp

```
#include <iostream>
#include <iomanip>
#include <chrono>
#include <cstring>
#include <vector>
#include <cstdint>

struct Results {
    float h2d_pageable_ms;
    float d2h_pageable_ms;
    float h2d_pinned_ms;
    float d2h_pinned_ms;
    float d2d_ms;

    int ok_h2d_d2h_pageable;
    int ok_h2d_d2h_pinned;
    int ok_d2d;
};

extern "C" Results run_memcpy_bench(size_t bytes, int iters, int do_warmup);

static double gbps(size_t bytes, double ms) {
    return (double)(bytes / 1e9) / (ms / 1000.0);
}

static void cpu_memcpy_bench(size_t bytes, int iters, bool pinned_like, double&
out_ms, bool& ok) {
    std::vector<uint8_t> a(bytes), b(bytes);
    for (size_t i = 0; i < bytes; i++) {
        a[i] = (uint8_t)((i * 17u + 3u) & 0xFF);
    }

    (void) pinned_like;
```

```
    auto t0 = std::chrono::high_resolution_clock::now();
    for (int i = 0; i < iters; i++) {
        std::memcpy(b.data(), a.data(), bytes);
    }
    auto t1 = std::chrono::high_resolution_clock::now();

    std::chrono::duration<double, std::milli> dt = t1 - t0;
    out_ms = dt.count() / iters;
    ok = (std::memcmp(a.data(), b.data(), bytes) == 0);
}

int main(int argc, char** argv) {
    size_t bytes = 256ull * 1024 * 1024; // 256 MiB default

    int iters  = 20;
    int warmup = 1;

    if (argc >= 2) {
        bytes = (size_t)std::stoull(argv[1]); // bytes
    }
    if (argc >= 3) {
        iters = std::stoi(argv[2]); // iterations
    }

    std::cout << "Buffer size: " << bytes << " bytes (" << std::fixed <<
std::setprecision(2) << (bytes / (1024.0*1024.0)) << " MiB)" << std::endl;
    std::cout << "Iterations: "  << iters << std::endl;

    // CPU -> CPU
    double cpu_ms = 0.0;
    bool cpu_ok = false;
    cpu_memcpy_bench(bytes, iters, false, cpu_ms, cpu_ok);

    std::cout << "[CPU->CPU memcpy] avg: " << cpu_ms << " ms, " << gbps(bytes,
cpu_ms) << " GB/s, ok=" << cpu_ok << std::endl;

    // GPU-related copies
    Results r = run_memcpy_bench(bytes, iters, warmup);

    auto pr = [&](const char* name, float ms) {
        std::cout << std::left << std::setw(28) << name
                  << " avg: " << std::right << std::setw(8) << std::fixed <<
std::setprecision(3)
                  << ms << " ms,  "
                  << std::setw(8) << std::setprecision(2) << gbps(bytes, ms)
                  << " GB/s" << std::endl;
    };

    pr("H->D pageable", r.h2d_pageable_ms);
    pr("D->H pageable", r.d2h_pageable_ms);
    std::cout << "  roundtrip ok=" << r.ok_h2d_d2h_pageable << std::endl;

    pr("H->D pinned", r.h2d_pinned_ms);
    pr("D->H pinned", r.d2h_pinned_ms);
    std::cout << "  roundtrip ok=" << r.ok_h2d_d2h_pinned << std::endl;

    pr("D->D", r.d2d_ms);
    std::cout << "  d2d ok=" << r.ok_d2d << std::endl;

    return 0;
}
```

Файл lab08_3.cu

```cpp
#include <cuda_runtime.h>
#include <cstdio>
#include <cstdint>
#include <cstdlib>
#include <cstring>

static void checkCuda(cudaError_t e) {
    if (e != cudaSuccess) {
        std::printf("CUDA error: %s\n", cudaGetErrorString(e));
        std::exit(1);
    }
}

static void fill_pattern(uint8_t* p, size_t n) {
    for (size_t i = 0; i < n; i++) p[i] = (uint8_t)((i * 1315423911u + 12345u) &
0xFF);
}

static bool equal_buf(const uint8_t* a, const uint8_t* b, size_t n) {
    return std::memcmp(a, b, n) == 0;
}

static float gbps(size_t bytes, float ms) {
    return (float)(bytes / 1e9) / (ms / 1000.0f);
}

struct Results {
    float h2d_pageable_ms;
    float d2h_pageable_ms;
    float h2d_pinned_ms;
    float d2h_pinned_ms;
    float d2d_ms;

    int ok_h2d_d2h_pageable;
    int ok_h2d_d2h_pinned;
    int ok_d2d;
};

extern "C" Results run_memcpy_bench(size_t bytes, int iters, int do_warmup) {
    Results r{};
    r.ok_h2d_d2h_pageable = 0;
    r.ok_h2d_d2h_pinned   = 0;
    r.ok_d2d              = 0;

    uint8_t* h_src = (uint8_t*)std::malloc(bytes);
    uint8_t* h_dst = (uint8_t*)std::malloc(bytes);
    uint8_t* h_chk = (uint8_t*)std::malloc(bytes);
    if (!h_src || !h_dst || !h_chk) {
        std::printf("malloc failed\n");
        std::exit(1);
    }

    uint8_t *hp_src = nullptr, *hp_dst = nullptr, *hp_chk = nullptr;
    checkCuda(cudaMallocHost((void**)&hp_src, bytes));
    checkCuda(cudaMallocHost((void**)&hp_dst, bytes));
    checkCuda(cudaMallocHost((void**)&hp_chk, bytes));

    uint8_t *d_a = nullptr, *d_b = nullptr;
    checkCuda(cudaMalloc((void**)&d_a, bytes));
```

```
    checkCuda(cudaMalloc((void**)&d_b, bytes));

    fill_pattern(h_src, bytes);
    std::memset(h_dst, 0, bytes);
    std::memset(h_chk, 0, bytes);

    fill_pattern(hp_src, bytes);
    std::memset(hp_dst, 0, bytes);
    std::memset(hp_chk, 0, bytes);

    // events
    cudaEvent_t e0, e1;
    checkCuda(cudaEventCreate(&e0));
    checkCuda(cudaEventCreate(&e1));

    auto timeMemcpy = [&](cudaMemcpyKind kind, const void* src, void* dst, float*
out_ms) {
        // warmup
        if (do_warmup) {
            checkCuda(cudaMemcpy(dst, src, bytes, kind));
            checkCuda(cudaDeviceSynchronize());
        }
        checkCuda(cudaEventRecord(e0));
        for (int i = 0; i < iters; i++) {
            checkCuda(cudaMemcpy(dst, src, bytes, kind));
        }
        checkCuda(cudaEventRecord(e1));
        checkCuda(cudaEventSynchronize(e1));

        float ms = 0.0f;
        checkCuda(cudaEventElapsedTime(&ms, e0, e1));
        *out_ms = ms / iters;
    };

    timeMemcpy(cudaMemcpyHostToDevice, h_src, d_a, &r.h2d_pageable_ms);
    timeMemcpy(cudaMemcpyDeviceToHost, d_a, h_dst, &r.d2h_pageable_ms);

    r.ok_h2d_d2h_pageable = equal_buf(h_src, h_dst, bytes) ? 1 : 0;

    timeMemcpy(cudaMemcpyHostToDevice, hp_src, d_a, &r.h2d_pinned_ms);
    timeMemcpy(cudaMemcpyDeviceToHost, d_a, hp_dst, &r.d2h_pinned_ms);

    r.ok_h2d_d2h_pinned = equal_buf(hp_src, hp_dst, bytes) ? 1 : 0;

    if (do_warmup) {
        checkCuda(cudaMemcpy(d_b, d_a, bytes, cudaMemcpyDeviceToDevice));
        checkCuda(cudaDeviceSynchronize());
    }
    checkCuda(cudaEventRecord(e0));
    for (int i = 0; i < iters; i++) {
        checkCuda(cudaMemcpy(d_b, d_a, bytes, cudaMemcpyDeviceToDevice));
    }
    checkCuda(cudaEventRecord(e1));
    checkCuda(cudaEventSynchronize(e1));
    float d2d_ms = 0.0f;
    checkCuda(cudaEventElapsedTime(&d2d_ms, e0, e1));
    r.d2d_ms = (d2d_ms / iters);

    checkCuda(cudaMemcpy(h_chk, d_b, bytes, cudaMemcpyDeviceToHost));
    r.ok_d2d = equal_buf(h_chk, h_src, bytes) ? 1 : 0;

    checkCuda(cudaEventDestroy(e0));
```

```
    checkCuda(cudaEventDestroy(e1));

    cudaFree(d_a);
    cudaFree(d_b);

    cudaFreeHost(hp_src);
    cudaFreeHost(hp_dst);
    cudaFreeHost(hp_chk);

    std::free(h_src);
    std::free(h_dst);
    std::free(h_chk);

    return r;
}
```

Результаты

Сборка и выполнение программы:

Выводы

- Во всех рассмотренных режимах копирования (CPU↔CPU, CPU↔GPU, GPU↔GPU) корректность подтверждена: данные после копирования совпадают с исходными (ok=1). Это означает, что выделение памяти и операции memcpy/cudaMemcpy выполнены правильно.

- Копирование между **оперативной памятью и глобальной памятью GPU** с использованием обычной (pageable) host-памяти показывает сравнительно низкую пропускную способность (≈6.3–6.5 GB/s в обоих направлениях). Это связано с тем, что pageable-память требует дополнительных действий со стороны драйвера (в т.ч. использование промежуточных буферов), что увеличивает накладные расходы.

- Использование **page-locked (pinned) памяти** существенно ускоряет обмен CPU↔GPU: пропускная способность выросла примерно до 13 GB/s, то есть примерно в 2 раза по сравнению с pageable-памятью. Следовательно, pinned-память является предпочтительной для интенсивного обмена данными между CPU и GPU.

- Копирование **между двумя буферами в глобальной памяти видеокарты (Device→Device)** оказалось самым быстрым (примерно 72 GB/s), что ожидаемо, так как операция выполняется внутри подсистемы памяти GPU и не ограничена пропускной способностью шины PCIe.

- Копирование **CPU→CPU** показало скорость порядка 9.6 GB/s и служит ориентиром для производительности подсистемы оперативной памяти; при этом скорость обмена CPU↔GPU ограничивается в основном интерфейсом подключения видеокарты и типом host-памяти.